

Paginação em Spring Data JPA

Guia prático com analogias, exemplos de código e boas práticas para APIs profissionais

Ideia central

Paginação evita que uma API busque milhares ou milhões de registros de uma vez. Em vez disso, ela entrega os dados em partes menores, como páginas de um livro.

1. Para Que Serve Paginação

Paginação serve para limitar a quantidade de registros retornados por uma consulta. Em um banco pequeno talvez um `findAll()` pareça inofensivo, mas em um banco real, com milhares ou milhões de linhas, retornar tudo de uma vez pode deixar a API lenta, consumir muita memória e pressionar o banco de dados.

```
@GetMapping
public List<Aluno> listar() {
    return alunoRepository.findAll();
}
```

Esse exemplo busca todos os alunos. Se existirem 500 mil alunos, a API tentará trazer 500 mil registros na mesma resposta.

Analogia

Sem paginação, seria como pedir ao arquivo do hospital todas as fichas de todos os pacientes de uma vez. Com paginação, você pede apenas as primeiras 20 fichas, depois as próximas 20, e assim por diante.

2. Como Funciona a Paginação

A API recebe parâmetros indicando qual página o usuário quer e quantos registros devem vir naquela página. No Spring, normalmente a contagem começa em zero.

Parâmetro	Exemplo	Significado
page	0	Primeira página
size	10	Quantidade de registros por página
sort	nome,asc	Ordenar pelo campo nome em ordem crescente

```
GET /alunos?page=0&size=10
GET /alunos?page=1&size=10
GET /alunos?page=0&size=10&sort=nome,asc
```

3. Pageable

Pageable é o objeto que carrega as informações da página: número da página, tamanho da página e ordenação. Você pode receber esse objeto diretamente no controller. O Spring monta o Pageable a partir dos query params da URL.

```
@GetMapping
public Page<Aluno> listar(Pageable pageable) {
    return alunoRepository.findAll(pageable);
}
```

Quando a requisição chega como `/alunos?page=0&size=10`, o Spring cria internamente um Pageable equivalente a: página 0, tamanho 10.

4. Page

Page é o resultado paginado. Ele não traz apenas a lista de registros. Ele também traz metadados, como total de elementos, total de páginas, página atual e tamanho da página.

```
{
  "content": [
    { "id": 1, "nome": "Pedro" },
    { "id": 2, "nome": "João" }
  ],
  "totalElements": 50,
  "totalPages": 5,
  "size": 10,
  "number": 0
}
```

Campo	Significado
content	Lista de registros da página atual
totalElements	Total de registros encontrados
totalPages	Quantidade total de páginas
size	Quantidade solicitada por página
number	Número da página atual

5. Exemplo Completo com Entity, DTO, Repository, Service e Controller

Entity

```
@Entity
@Table(name = "alunos")
public class Aluno {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    private String nome;
    private String email;
}
```

DTO de resposta

```
public record AlunoResponseDTO(
    Long id,
    String nome,
    String email
) {
}
```

Repository

```
public interface AlunoRepository extends JpaRepository<Aluno, Long> {
}
```

Service

```

@Service
public class AlunoService {

    private final AlunoRepository alunoRepository;

    public AlunoService(AlunoRepository alunoRepository) {
        this.alunoRepository = alunoRepository;
    }

    public Page<AlunoResponseDTO> listar(Pageable pageable) {
        return alunoRepository.findAll(pageable)
            .map(aluno -> new AlunoResponseDTO(
                aluno.getId(),
                aluno.getNome(),
                aluno.getEmail()
            ));
    }
}

```

Controller

```

@RestController
@RequestMapping("/alunos")
public class AlunoController {

    private final AlunoService alunoService;

    public AlunoController(AlunoService alunoService) {
        this.alunoService = alunoService;
    }

    @GetMapping
    public ResponseEntity<Page<AlunoResponseDTO>> listar(Pageable pageable) {
        return ResponseEntity.ok(alunoService.listar(pageable));
    }
}

```

6. Ordenação

A ordenação pode ser enviada pela URL usando sort. Isso ajuda muito em telas de listagem, onde o usuário quer ver por nome, data de cadastro, maior valor, menor valor e assim por diante.

```

GET /alunos?page=0&size=10&sort=nome,asc
GET /alunos?page=0&size=10&sort=id,desc

```

asc significa crescente. desc significa decrescente.

7. Paginação Padrão com @PageableDefault

Você pode definir um comportamento padrão para quando o usuário não informar page, size ou sort. Isso deixa a API mais previsível.

```

@GetMapping
public ResponseEntity<Page<AlunoResponseDTO>> listar(
    @PageableDefault(size = 10, sort = "nome") Pageable pageable) {

    return ResponseEntity.ok(alunoService.listar(pageable));
}

```

Agora, se o usuário chamar apenas GET /alunos, o Spring usa size 10, page 0 e ordenação por nome.

8. Filtro com Paginação

A paginação também funciona junto com filtros. Exemplo: buscar alunos pelo nome, mas retornando somente uma página de cada vez.

```
public interface AlunoRepository extends JpaRepository<Aluno, Long> {  
  
    Page<Aluno> findByNomeContainingIgnoreCase(String nome, Pageable pageable);  
}
```

```
public Page<AlunoResponseDTO> buscarPorNome(String nome, Pageable pageable) {  
    return alunoRepository.findByNomeContainingIgnoreCase(nome, pageable)  
        .map(aluno -> new AlunoResponseDTO(  
            aluno.getId(),  
            aluno.getNome(),  
            aluno.getEmail()  
        ));  
}
```

```
@GetMapping("/buscar")  
public ResponseEntity<Page<AlunoResponseDTO>> buscarPorNome(  
    @RequestParam String nome,  
    @PageableDefault(size = 10, sort = "nome") Pageable pageable) {  
  
    return ResponseEntity.ok(alunoService.buscarPorNome(nome, pageable));  
}
```

```
GET /alunos/buscar?nome=pedro&page=0&size=10
```

9. PageRequest

PageRequest é usado quando você quer criar um Pageable manualmente dentro do código, sem depender diretamente dos parâmetros da URL.

```
Pageable pageable = PageRequest.of(  
    0,  
    10,  
    Sort.by("nome").ascending()  
);  
  
Page<Aluno> pagina = alunoRepository.findAll(pageable);
```

Isso cria uma consulta para a primeira página, com 10 registros, ordenada por nome.

10. Page vs List vs Slice

Retorno	Quando usar	Observação
List	Consultas pequenas e internas	Não traz metadados de paginação
Page	Endpoints de listagem em APIs	Traz total de elementos e total de páginas
Slice	Quando basta saber se existe próxima página	Pode ser mais leve que Page em grandes volumes

O Page geralmente executa uma consulta para buscar os dados e outra para contar o total. Em tabelas gigantes, essa contagem pode custar caro. Quando o total não importa, Slice pode ser uma escolha melhor.

```
// Ideia aproximada do que acontece com Page:  
SELECT * FROM alunos ORDER BY nome LIMIT 10 OFFSET 0;  
SELECT COUNT(*) FROM alunos;
```

11. Como Isso Chega no Banco

Quando usamos paginação, o Spring Data JPA gera uma consulta com limite e deslocamento. Em PostgreSQL, por exemplo, costuma aparecer como LIMIT e OFFSET. Em Oracle, o SQL pode usar FETCH FIRST, OFFSET ou subconsultas, dependendo da versão e do dialeto configurado.

```
-- Exemplo conceitual em PostgreSQL
SELECT *
FROM alunos
ORDER BY nome ASC
LIMIT 10 OFFSET 0;
```

OFFSET 0 significa começar do primeiro registro. Se a página for 1 e o tamanho 10, o offset será 10. Se a página for 2, offset 20.

12. Boas Práticas Profissionais

Boa prática	Por que importa
Não retornar tudo	Evita lentidão e consumo alto de memória
Usar DTO	Evita expor a entidade e dados sensíveis
Definir size padrão	Evita respostas gigantes por acidente
Limitar tamanho máximo	Impede chamadas como size=100000
Ordenar por campos indexados	Melhora performance em tabelas grandes
Evitar SELECT * em consultas nativas	Retorna apenas o necessário
Avaliar Slice em alto volume	Evita contagem total quando ela não é necessária

13. Exemplo de Limite Manual de Size

Uma preocupação real: o usuário pode chamar /alunos?size=100000. Para evitar isso, você pode limitar no service ou configurar globalmente. Um exemplo simples no controller/service seria normalizar o Pageable.

```
private Pageable limitarTamanho(Pageable pageable) {
    int tamanhoMaximo = 100;
    int tamanho = Math.min(pageable.getPageSize(), tamanhoMaximo);

    return PageRequest.of(
        pageable.getPageNumber(),
        tamanho,
        pageable.getSort()
    );
}
```

Assim, mesmo que alguém peça 100000 registros, a aplicação limita para 100.

14. Resumo Final

Conceito	Resumo
Paginação	Buscar dados em partes menores
Pageable	Pedido de paginação: page, size e sort

Conceito	Resumo
Page	Resposta com dados e metadados
@PageableDefault	Define valores padrão
PageRequest	Cria paginação manualmente no Java
DTO + Page	Formato recomendado para APIs
Banco grande	Paginação deixa a API mais segura e performática

Frase para fixar

Paginação é como entregar um livro por capítulos. O banco tem o livro inteiro, mas a API entrega apenas o capítulo que o usuário pediu.